

Online supplement to:
*Completion-aware search with partial witness repair for dense
fixed-shape facility layout*

Junhwan Ahn Kunwoo Lee Kyunghyun Lee Yeoneung Kim

This supplement reports the experimental details supporting partial witness repair. Section S1 gives the primary 60-second comparison of M0, M1, and ALNS: control selection, initialization coverage, quality curves, target attainment, and timing. Section S2 reports the one-second mechanism study, which isolates reuse, repair, and final verification.

Sections S3 to S10 provide secondary comparisons of transfer, quality and diversity, amortized cost, action granularity, reward configurations, completion checks, and constructor feasibility. Section S11 examines feasible search space, initialization, and candidate selection. The secondary evaluation protocol and dataset coverage are specified in Section S12; policy architecture and training costs are in Section S13. The two matched-time studies and the secondary experiments are analyzed under their respective protocols, without pooling incompatible timing records.

S1 Comparison through 60 seconds

This study evaluates M0, M1, and an adaptive repair control on a common initialized set. It crosses $n = 32, 64, 128, 256$, guillotine and nonslicing geometry, and nominal fills 0.90 and 0.95, with 10 and 20 attempted instances per geometry and size cell at the respective fills. Instance tags are disjoint from S2, and these timing records are analyzed separately. The maximum plate side is 44, 44, 72, and 100 for the four sizes; sampled mean-area bounds remain 9 to 30. Integer dimensions make fill approximate. The generated source packing is verified but withheld from every search.

S1.1 Adaptive repair control and validation

The control is a layout-specific ALNS adaptation (Ropke & Pisinger, 2006). It combines three removal rules with two insertion rules. Random removal samples facilities uniformly without replacement. Spatial removal selects facilities with centers nearest a random anchor. Low-contribution removal ranks facilities by incident pair and boundary rewards and samples without replacement with weights proportional to the inverse square root of rank. The constant no-overlap bonus is omitted from these contribution scores. The removal count is uniform from two to the selected cap.

Beam insertion visits removed facilities in decreasing area order, shuffled with probability 0.25, and keeps a bounded frontier ranked by accumulated exact marginal gain. Regret insertion repeatedly inserts the facility with the largest gap between its two best legal positions, breaking ties by area. Both use the configured number of ranked candidate positions and append

the facility’s original position when legal. Every complete candidate is checked by the same independent verifier and objective evaluator as M0 and M1.

The six operator-pair weights start at one. After an attempt, the selected weight is updated to $\max(0.2, 0.9w + 0.1r)$, where the reward is six for a new best layout, three for an accepted improving move of the current layout, one for an accepted nonimproving move, and zero otherwise. The best layout is retained separately from the current layout. A nonimproving move with change ΔJ is accepted with probability $\exp(\Delta J/T)$. The initial temperature is $0.0025 \max(|J(w_0) - \beta_{\max}|, 1)$ and is multiplied by 0.999 per attempt. These online updates are fixed parts of the search rule.

Validation uses 16 disjoint instances: two per geometry, fill, and size cell at $n = 64$ and 128, with two seeds and a 10-second cutoff. The configuration with the largest mean improvement after averaging seeds within instance is selected before testing. Table S1 reports all four candidates and the B2/B2S reference controls. The selected ALNS configuration has removal cap four, beam width four, and four ranked candidates. M0 and M1 retain $\kappa = 32$, one completion attempt, and M1’s repair cap of four.

Table S1: Validation at 10 seconds on 16 instances with two seeds. Improvement is the mean percentage gain over the common initializer. The selected ALNS configuration has removal cap four and beam width four. Candidate counts exclude the extra original position when legal; B2 and B2S use a different insertion rule.

method	removal cap	beam width	candidates	improvement
ALNS	4	4	4	7.80
ALNS	8	4	4	7.63
ALNS	8	8	8	7.08
ALNS	16	4	4	7.15
B2	16	n/a	n/a	5.83
B2S	4	n/a	n/a	6.99

S1.2 Coverage and quality

Table S2 includes all 240 attempted instances. Initialization succeeds on 236; the four failures are split equally between the two $n = 32$, fill 0.95 cells and are not replaced. Three methods and three seeds give 2,124 completed runs. Each run continues for 60 seconds and records cutoffs 0, 0.1, 0.3, 1, 3, 10, 30, and 60 seconds.

Tables S3 and S4 give geometry-specific quality at the primary cutoffs. The full set of nonzero cutoffs is shown in Figure S1. Improvement uses $100[J_m(t) - J(w_0)]/J(w_0)$, including the constant feasible-layout bonus in the denominator. Three seeds are averaged within each instance. Pooled contrasts give equal weight to the two geometry families, with 10,000 paired bootstrap samples resampling instances within geometry. All 95% intervals are descriptive and pointwise, without multiplicity adjustment. Raw objective gains, every cutoff, and geometry-specific paired intervals are retained in the accompanying result tables.

S1.3 Time to a target improvement

For each run and target $q \in \{1, 5, 10\}\%$, the first verified trace event reaching q defines the hitting time. A non-hit contributes the 60-second cutoff to the restricted mean. Table S5 reports

Table S2: Initialization coverage and achieved fill in the 60-second study. Failures remain in the attempted count. Fill ranges cover all attempted instances; mean initialization-attempt time is in seconds over all attempts.

geometry	n	fill	initialized	achieved fill	init. time
guillotine	32	0.90	10/10	0.89815 to 0.90000	0.0055
guillotine	32	0.95	18/20	0.94318 to 0.95000	0.0052
guillotine	64	0.90	10/10	0.89855 to 0.89959	0.0128
guillotine	64	0.95	20/20	0.94792 to 0.95000	0.0153
guillotine	128	0.90	10/10	0.89938 to 0.90000	0.0332
guillotine	128	0.95	20/20	0.94885 to 0.95000	0.0299
guillotine	256	0.90	10/10	0.89965 to 0.90000	0.0999
guillotine	256	0.95	20/20	0.94943 to 0.95000	0.0970
nonslicing	32	0.90	10/10	0.89583 to 0.90000	0.0044
nonslicing	32	0.95	18/20	0.94603 to 0.95000	0.0036
nonslicing	64	0.90	10/10	0.89836 to 0.90000	0.0099
nonslicing	64	0.95	20/20	0.94796 to 0.94994	0.0104
nonslicing	128	0.90	10/10	0.89890 to 0.90000	0.0274
nonslicing	128	0.95	20/20	0.94915 to 0.95000	0.0304
nonslicing	256	0.90	10/10	0.89942 to 0.90000	0.0905
nonslicing	256	0.95	20/20	0.94933 to 0.95000	0.0873

Table S3: Geometry-specific mean percentage improvement at fill 0.90. Seeds are averaged within each instance before the cell mean. Time excludes initialization; all methods use the common initialized set.

geometry	n	seconds	M0	M1	ALNS
guillotine	32	10	25.08	21.96	20.00
guillotine	32	60	28.19	26.29	25.59
guillotine	64	10	17.49	14.80	12.95
guillotine	64	60	18.85	17.57	19.31
guillotine	128	10	16.20	14.31	8.52
guillotine	128	60	18.02	15.49	15.61
guillotine	256	10	8.66	13.44	3.28
guillotine	256	60	19.95	14.72	10.06
nonslicing	32	10	18.93	18.86	18.80
nonslicing	32	60	20.67	22.36	21.75
nonslicing	64	10	16.28	15.01	14.14
nonslicing	64	60	17.70	17.41	19.14
nonslicing	128	10	17.44	14.89	10.61
nonslicing	128	60	19.63	16.98	17.68
nonslicing	256	10	10.74	13.93	4.10
nonslicing	256	60	18.84	14.67	11.47

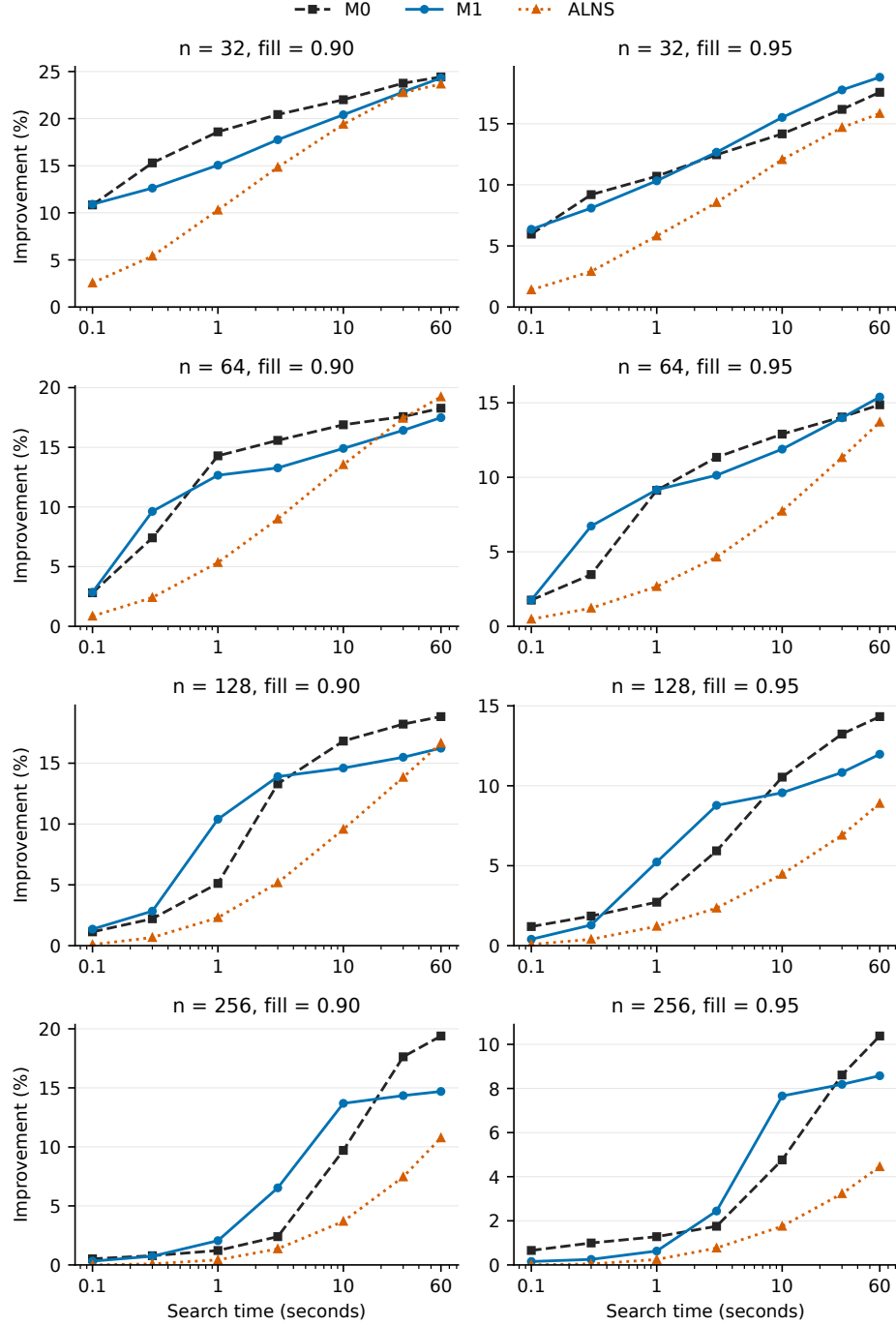


Figure S1: All size and fill cells of the 60-second comparison. Curves average seeds within instance and then give the two geometries equal weight. Connecting lines guide the eye between recorded cutoffs; they are not independently restarted searches. The common initialization time is excluded.

Table S4: Geometry-specific mean percentage improvement at fill 0.95. Seeds are averaged within each instance before the cell mean. Time excludes initialization; all methods use the common initialized set.

geometry	n	seconds	M0	M1	ALNS
guillotine	32	10	13.60	15.60	11.87
guillotine	32	60	17.72	19.38	16.25
guillotine	64	10	13.90	11.87	7.78
guillotine	64	60	16.08	15.90	13.99
guillotine	128	10	11.56	9.61	4.45
guillotine	128	60	15.50	12.07	9.14
guillotine	256	10	5.71	8.00	1.75
guillotine	256	60	11.80	9.12	4.34
nonslicing	32	10	14.74	15.44	12.26
nonslicing	32	60	17.44	18.24	15.43
nonslicing	64	10	11.89	11.91	7.67
nonslicing	64	60	13.65	14.87	13.41
nonslicing	128	10	9.52	9.52	4.48
nonslicing	128	60	13.15	11.88	8.65
nonslicing	256	10	3.82	7.32	1.76
nonslicing	256	60	8.96	8.03	4.56

both hit percentage and restricted mean, pooling geometries equally. A companion total-time column in the result archive adds each run’s measured initialization time. These summaries are descriptive, with equal seed counts per instance; no success-only timing average is used.

At $n = 256$, fill 0.95, M1 reaches a 5% improvement in 94.2% of runs with a restricted mean of 9.76 seconds, compared with 90.0% and 20.73 seconds for M0. At the 10% target, the order reverses: M0 reaches the target in 56.7% of runs with a restricted mean of 39.15 seconds, while M1 reaches it in 20.0% with 51.74 seconds. This separates early moderate improvement from attainment of a more demanding target.

S1.4 Timing and independent checks

Validation and test use eight single-threaded processes pinned to distinct physical cores of an AMD Ryzen 7 9700X, with Python 3.10.18 and NumPy 2.2.5 on Windows 10. All methods and seeds for one instance run on the same core in deterministic shuffled order. BLAS and OpenMP thread counts are one. Table S6 reports the full-run CPU/wall ratios and return overruns. No low-ratio run is removed. These are throughput measurements under the stated allocation, rather than single-worker latency measurements.

The independent audit regenerates all attempted instances, checks their source packings and common initializations, verifies all 17,228 saved layouts, and recomputes every saved objective. It also checks the expected run identities, trace monotonicity, and the eligibility of every saved budget row. All checks pass, with zero objective recomputation error. The 130 completed events finishing after the final deadline are discarded. Protocol and result-manifest hashes link the reported tables to these audited records.

Table S5: Target attainment within 60 seconds, pooling the two geometries with equal weight. Each entry is hit percentage / restricted mean search time in seconds. Non-hits contribute 60 seconds, so the time is not a success-only mean. Three seeds per instance contribute equally.

n	fill	method	target 1%	target 5%	target 10%
32	0.90	M0	100.0 / 0.02	100.0 / 0.05	100.0 / 0.12
32	0.90	M1	100.0 / 0.02	100.0 / 0.04	100.0 / 0.23
32	0.90	ALNS	100.0 / 0.06	100.0 / 0.37	100.0 / 1.81
64	0.90	M0	100.0 / 0.07	100.0 / 0.22	83.3 / 11.19
64	0.90	M1	100.0 / 0.05	100.0 / 0.51	95.0 / 6.15
64	0.90	ALNS	100.0 / 0.16	100.0 / 1.69	95.0 / 10.23
128	0.90	M0	100.0 / 0.21	100.0 / 1.17	93.3 / 7.35
128	0.90	M1	100.0 / 0.16	100.0 / 0.58	90.0 / 7.01
128	0.90	ALNS	100.0 / 0.46	100.0 / 3.79	95.0 / 16.17
256	0.90	M0	100.0 / 1.37	100.0 / 7.10	100.0 / 11.71
256	0.90	M1	100.0 / 0.64	100.0 / 2.58	100.0 / 5.30
256	0.90	ALNS	100.0 / 3.01	96.7 / 20.55	53.3 / 49.75
32	0.95	M0	100.0 / 0.06	100.0 / 0.65	88.0 / 12.76
32	0.95	M1	100.0 / 0.04	100.0 / 0.71	93.5 / 7.92
32	0.95	ALNS	100.0 / 0.13	99.1 / 2.80	83.3 / 17.19
64	0.95	M0	100.0 / 0.25	95.8 / 4.14	77.5 / 18.11
64	0.95	M1	100.0 / 0.11	100.0 / 0.37	90.8 / 14.55
64	0.95	ALNS	100.0 / 0.34	100.0 / 6.92	71.7 / 31.12
128	0.95	M0	100.0 / 1.87	94.2 / 7.48	72.5 / 23.71
128	0.95	M1	100.0 / 0.55	95.8 / 5.73	65.8 / 28.08
128	0.95	ALNS	100.0 / 1.21	87.5 / 22.48	31.7 / 51.49
256	0.95	M0	100.0 / 4.51	90.0 / 20.73	56.7 / 39.15
256	0.95	M1	100.0 / 1.96	94.2 / 9.76	20.0 / 51.74
256	0.95	ALNS	100.0 / 6.25	38.3 / 54.54	0.0 / 60.00

Table S6: Timing over 708 runs per method. Ratios compare process CPU time with wall time. Return overruns are milliseconds beyond the 60-second deadline. Late completed candidates are discarded, not admitted to any quality comparison. Final verification failures are zero for all methods.

method	mean ratio	min. ratio	median ratio	mean overrun	max. overrun	late
M0	0.9798	0.6898	0.9970	5.00	61.06	42
M1	0.9794	0.6953	0.9971	3.08	43.61	74
ALNS	0.9805	0.6515	0.9973	8.44	66.43	14

S2 One-second mechanism study

Every row below uses a common verified best-contact layout and retains it as a feasible incumbent. Stochastic seeds are averaged within an instance before cell means are computed. The 0.1 and 0.3 second values are read from the same anytime run that continues to 1.0 second. Times exclude the common measured initialization cost, which is charged equally to all methods in total-time reporting. Mean initialization and verification time ranges from 0.0052 to 0.0128 seconds over the four cells. All normalized improvements use the full feasible objective J_β , including the constant no-overlap bonus.

Table S7: One-second dense study at fill 0.95. Entries are mean percentage improvement over the common initial layout.

geometry	n	sec.	B1	B2	B2S	B3	M0	R0	M1
guillotine	32	0.1	1.72	0.55	0.82	2.45	3.50	1.64	6.14
		0.3	1.72	1.22	1.82	4.37	7.59	1.96	7.83
		1.0	1.72	2.40	3.51	9.63	9.04	2.43	9.51
guillotine	64	0.1	0.00	0.22	0.41	1.29	1.72	0.21	1.41
		0.3	4.20	0.68	0.87	2.21	3.64	1.33	5.21
		1.0	4.44	1.41	1.62	3.69	8.00	2.31	8.76
nonslicing	32	0.1	1.79	0.57	1.02	1.95	3.50	1.91	6.32
		0.3	1.79	1.43	2.03	4.13	7.63	2.27	7.83
		1.0	1.79	2.64	3.71	9.53	10.15	2.86	9.65
nonslicing	64	0.1	0.00	0.18	0.44	1.64	1.47	0.13	1.14
		0.3	3.46	0.74	0.96	2.31	2.56	1.46	4.68
		1.0	4.10	1.38	1.78	3.60	7.78	2.70	7.98

Table S8: Pooled M1 minus control contrasts on 196 initialized dense instances. Entries are percentage points with paired instance-bootstrap 95% intervals.

sec.	B1	B2	B2S	B3	M0	R0
0.1	2.84 [2.26,3.45]	3.32 [2.83,3.82]	3.03 [2.56,3.52]	1.87 [1.36,2.39]	1.18 [0.58,1.75]	2.75 [2.33,3.17]
0.3	3.55 [2.57,4.49]	5.35 [4.84,5.86]	4.95 [4.47,5.43]	3.12 [2.58,3.67]	1.05 [0.46,1.67]	4.61 [4.12,5.10]
1.0	5.93 [4.99,6.88]	7.02 [6.54,7.52]	6.33 [5.84,6.82]	2.41 [1.74,3.06]	0.25 [-0.42,0.92]	6.39 [5.89,6.92]

At 1.0 second, the raw M1 minus B1 difference is 224.5 objective units [172.2,276.5], its win/tie/loss count is 156/0/40, and 152 of 196 instances exceed the one-percent practical threshold. The corresponding counts against B2, B2S, M0 and R0 are 196/0/0, 195/0/1, 109/0/87 and 195/0/1. The M0 interval does not establish a difference.

Across all dense M1 runs, the repair branch produces 6,215 incumbent updates and 142,270 objective units of logged gain; unchanged reuse produces 1,380 updates and 12,455 units. These event counts are mechanism descriptions rather than inferential samples. B2 and B2S have mean destroy sizes 8.88 to 9.06 and 2.99, respectively.

All 8,820 dense budget layouts and 5,214 boundary budget layouts pass an independent result audit. Recomputed objective and trace selection errors are zero. No admitted dense event is later than 0.999998 seconds. Nonpreemptive work can return up to 9.8 ms after the cutoff, but its candidate is not admitted. The adversarial regression set rejects duplicate and missing facilities, negative and out-of-range coordinates, overlap, stale witnesses and incomplete timeout returns, while accepting legal edge contact.

Table S9: Boundary study at fill 0.90 and 1.0 second. Entries are mean percentage improvement over the common initial layout. The last column is the paired M1 minus B1 percentage-point difference with a 95% bootstrap interval.

geometry	n	init.	B1	B2	B2S	B3	M0	R0	M1	M1-B1
guillotine	32	20/20	13.03	7.89	7.60	8.84	16.17	7.97	14.60	1.57 [-0.27,3.65]
guillotine	64	20/20	14.06	5.53	4.94	4.70	15.48	6.25	13.44	-0.62 [-2.09,1.07]
nonslicing	32	19/20	11.86	6.92	7.28	9.43	14.90	8.03	13.84	1.97 [-0.07,4.02]
nonslicing	64	20/20	14.98	5.01	4.82	4.10	15.85	6.68	13.07	-1.91 [-3.54,-0.25]

Table S10: Mechanism log at fill 0.95 and 1.0 second. Repair success is conditional on a nonempty displaced set with size at most four. Verified completions exclude the carried fallback. R0 contains reuse only, and M0 uses full regeneration for every nonfallback proposal.

geometry	n	mean $ D $	repair success	M1 reuse	M1 verified	R0 verified	M0 verified
guillotine	32	1.95	2.45%	14.0	80.8	26.4	59.6
guillotine	64	2.49	3.91%	6.4	61.1	23.8	40.9
nonslicing	32	2.05	2.19%	15.9	76.1	29.7	59.0
nonslicing	64	2.72	3.55%	6.9	55.0	26.2	35.7

Table S11: Mean measured work per one-second dense run, in milliseconds. Late is the number of candidates that completed after the cutoff and were discarded.

method	generation	intermediate verify	final verify	objective	return time	late
B1	0.0	0.0	0.1	0.1	1001.53	0
B2	0.0	0.0	2.9	2.6	1002.58	1
B2S	0.0	0.0	15.6	13.5	1002.67	14
B3	845.4	37.7	40.3	40.9	1001.44	28
M0	719.3	21.4	30.7	30.6	1001.26	23
R0	0.0	12.6	105.0	92.9	1001.66	81
M1	358.1	30.3	65.2	61.8	1001.11	47

S3 Zero shot transfer

A trained policy can have an advantage through amortization. A solver restarts for every brief, while a policy evaluates a new brief in one forward pass. We trained a policy only on procedurally generated briefs. The type inventory, dimensions, affinity tables, plate aspect ratio, fill ratio, wall preferences and objective constants were randomized during 1.2M training steps. We then evaluated the policy on briefs that were not optimized during training.

Table S12: Zero shot transfer at $n = 32$. The greedy constructor provides the direct comparison because, like the policy, it does not search over the plate. It also requires no training. Its cost is 32 and 528 marginal objective sweeps per layout; the batched figure on the heterogeneous generated set is higher, so we report the cost for one instance.

test set	shelf	SA 45k lat.	greedy constructive		RL zero shot ($\times 16 + \text{pol.}$)
			area ord.	room+pos.	
generated (in distribution)	1 912.6	2 613.9	2 488.8	2 597.5	2 591.0
OFFICE (unseen)	2 942.5	3 412.1	3 359.5	3 380.2	3 426.3
CLINIC (unseen)	3 503.7	4 011.9	3 955.6	4 116.5	3 950.2
evaluations/layout	1	58 963 to 65 875	32	528	2 049 to 5 633

On OFFICE the zero shot policy reaches 3 414.5 with a single greedy decode (32 evaluations, 12.7 ms), matching annealing’s 3 412.1 at 60 371 evaluations, and 3 426.3 with 16 samples and a polish. At the same 32 evaluations the constructor without training reaches 3 359.5.

CLINIC gives a different result. The greedy constructor reaches 4 116.5, which is higher than both full budget annealing and the zero shot policy. At a matched 32 evaluations, it reaches 3 955.6 against the policy’s 3 848.5. Across the three test sets the zero shot policy is better on one, similar on one and worse on one. These results show that one set of weights can evaluate unseen briefs with a quality similar to large budget annealing while using three to four orders of magnitude less search, with the relative ranking varying across briefs. Transfer is evaluated at a fixed facility count because the policy’s input dimension depends on n (Section S12.2).

S4 Quality and diversity

A common argument for learning in design is that a policy provides a *distribution* over layouts. To compare quality and diversity across all three methods, we vary the sampling temperature for the policy, the budget for annealing, and the fraction of randomized commitments for the constructor without training.

We make three observations. First, a policy whose entropy bonus is annealed to zero produces almost no diversity. At $T=1$, its diversity is 0.006. Second, randomizing the commit order increases diversity by a factor of fifty with a 0.6% decrease in objective. Third, applying the same procedure to the greedy rule produces its own front. Randomizing a controlled fraction of the commit slots changes diversity continuously from 0.000 to 0.620 and reaches 3 415.2 at diversity 0.531 for 32 objective evaluations, which matches the best annealing point anywhere on the front (3 414.5 at 0.639, 61 267 evaluations).

The two fronts cross once. The policy has a higher objective up to diversity ≈ 0.42 , while the constructor without training has a higher objective above ≈ 0.45 . Partial randomization of the commit order resolves the interval between 0.000 and 0.603 left between the deterministic

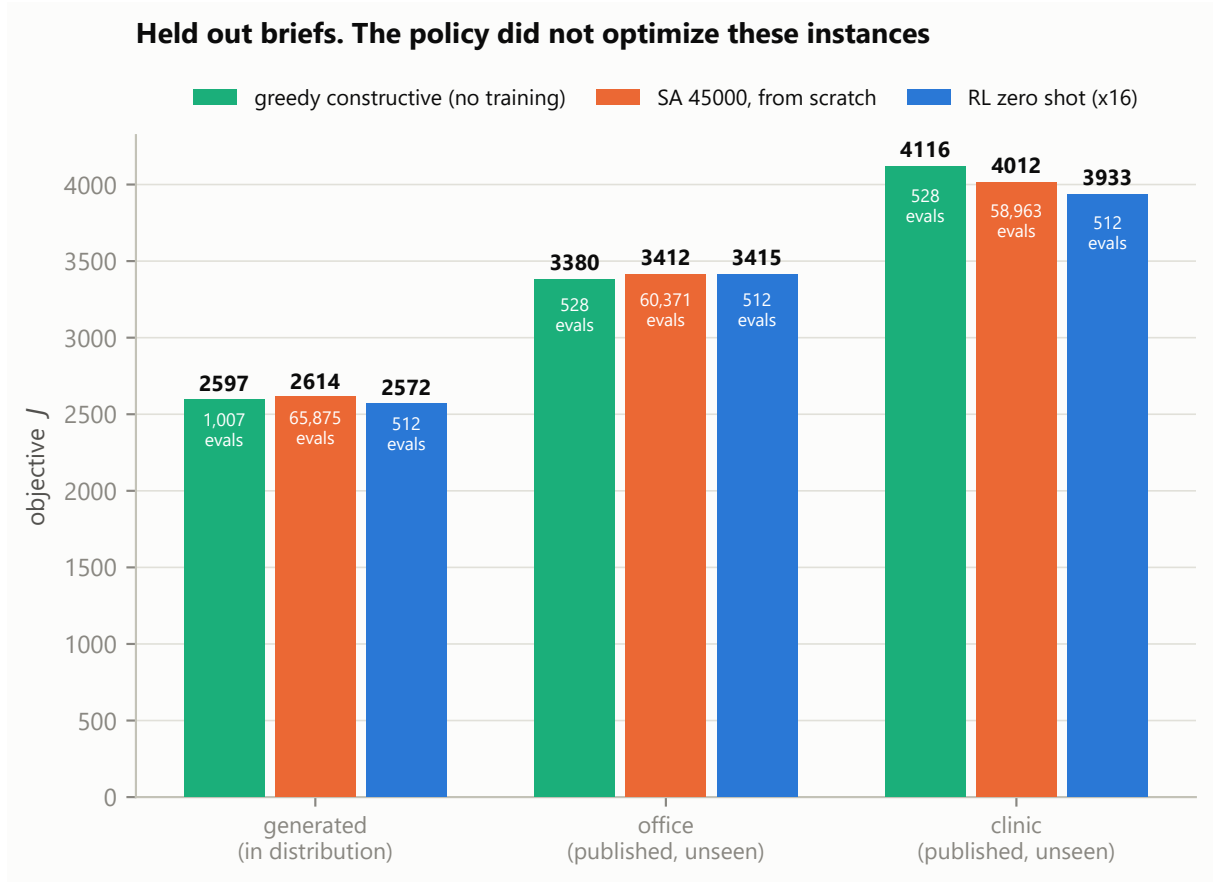


Figure S2: Held out briefs, annotated with objective evaluations per layout. The two reference scenarios were authored by hand and never optimized in training. The RL bars show 16 samples without the final polish (512 evaluations), a cheaper operating point than the $\times 16$ +polish column of Table S12, which is why the two differ slightly.

Table S13: Quality against diversity on OFFICE, 64 samples from one start. Diversity is the mean pairwise fraction of cells whose room type differs, estimated from 400 of the 2016 pairs, so it carries about ± 0.01 of sampling noise; every row uses the same estimator.

method	J	diversity	ms/sample
policy, order chosen by policy, $T=1.0$	3 529.3	0.006	22
policy, random commit order, $T=1.0$	3 508.0	0.304	19
$T=2.5$	3 418.2	0.405	18
$T=5.0$	3 048.0	0.623	13
SA 8 192, lattice	3 321.1	0.581	478
SA 45 000, lattice	3 414.5	0.639	1 070
greedy constructive, area order	3 359.5	0.000	7
20% of commit slots reshuffled	3 374.5	0.350	114
50%	3 408.3	0.482	251
70%	3 415.2	0.531	294
100%	3 405.2	0.620	347

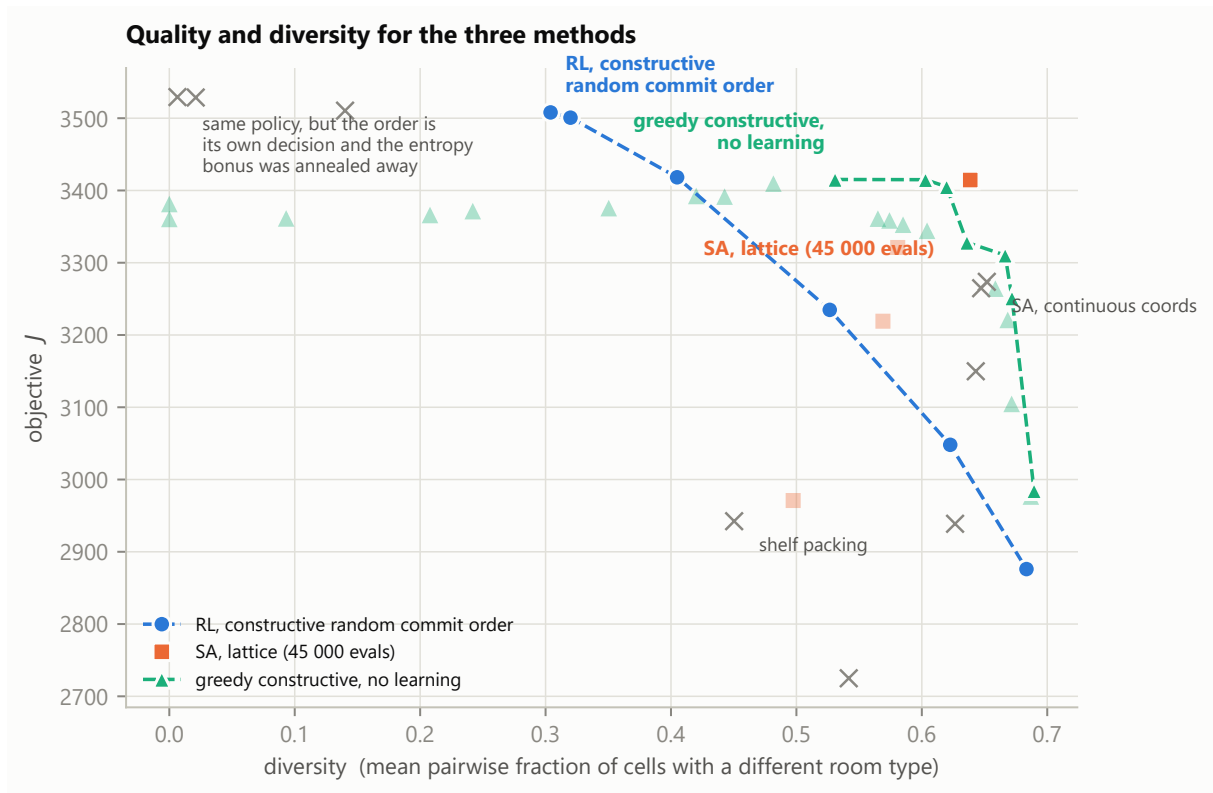


Figure S3: Quality against diversity with all three sides swept. Faint markers are points that a method's own front dominates.

and uniformly randomized settings. This comparison covers a single scenario; evaluation across the generated suite remains future work.

S5 Amortized cost

Training has a fixed initial cost, while search has a cost for every layout. Table S14 reports the annealing budget sweep on OFFICE, each budget’s wall clock time per layout, and the first point on the policy’s training curve (488 recorded checkpoints, 2 049 s total) at which the policy’s greedy decode exceeds that budget’s mean objective. The policy finishes at 3 529.3 and decodes in 17.8 ms per layout.

Table S14: Annealing quality, evaluation time, and policy training time needed to exceed each mean objective. The 45k lattice row uses the scenario comparison; the 150k to 10^6 rows use the budget sweep. These are separate measurements: the sweep’s 45k run gives 3 400.8 at 1.13 s. Timing is not reported for the non-batched reference annealer.

annealer	mean J	s/layout	training s to exceed
non-batched reference, 45k	3 269.9	not reported	109
lattice, 45k	3 414.8	1.00	157
lattice, 150k	3 459.0	3.40	215
lattice, 400k	3 489.1	8.83	256
lattice, 10^6	3 501.3	20.08	268

Charging the full 2 049 s up front and 17.8 ms per layout thereafter, the policy has a lower total wall clock cost than annealing with 10^6 evaluations after $2049/(20.08 - 0.018) \approx 102$ layouts. Compared with lattice annealing using 45 000 evaluations, this occurs after $2049/(1.00 - 0.018) \approx 2 093$ layouts.

This comparison has two limitations. First, the workload contains repeated requests for the *same* brief. An annealer could store its best layout and reuse it for later requests. A stream of unseen briefs is evaluated separately in S3. Second, this comparison is against annealing and not against the constructor without training. At 88 ms per layout, the initial training cost is recovered only after roughly 29 000 layouts relative to the constructor. For smaller workloads, this constructor has the lower total cost.

S6 Action granularity

We compare a discrete action head for small moves with a continuous displacement head when feasibility is handled by a penalty. We report the comparison inside the improvement MDP on the single $n = 32$ OFFICE scenario (best observed reward, jump and swap moves, Metropolis acceptance). One checkpoint per action head is evaluated from the same 64 independent random starts, shown in Table S15. No tradeoff appears in this evaluation. With sampled decoding at horizon 4 096 the lattice head reaches 3 069.6 at 50% feasibility while the continuous head reaches 2 737.0 at 0%, and deterministic decoding of either head falls into a limit cycle and loses more than a thousand objective units. The feasible-volume record in Table S18 explains the direction geometrically, and under the constructive formulation the question does not arise.

Table S15: Discrete against continuous action head inside the improvement MDP with best observed reward, jump and swap moves, and Metropolis acceptance on OFFICE, $n = 32$: mean and standard deviation across the same 64 independent evaluation starts.

head	decode	horizon	J	feasible
discrete (lattice)	sampled	512	$2\,475.3 \pm 93.8$	0.00
discrete (lattice)	sampled	2\,048	$2\,853.0 \pm 101.4$	0.06
discrete (lattice)	sampled	4\,096	$3\,069.6 \pm 126.3$	0.50
discrete (lattice)	argmax	4\,096	$1\,372.6 \pm 228.4$	0.00
continuous	sampled	512	$2\,354.7 \pm 76.2$	0.00
continuous	sampled	2\,048	$2\,611.4 \pm 85.5$	0.00
continuous	sampled	4\,096	$2\,737.0 \pm 60.8$	0.00
continuous	mean	4\,096	$1\,649.5 \pm 203.9$	0.00

S7 Reward and action ablation

Table S16 compares reward, action, and acceptance configurations of the penalty-based improvement MDP on OFFICE. Each configuration is trained for 688k environment steps and evaluated on 64 held out starts using stochastic decoding at 4\,096 objective evaluations. The configurations distinguish single-facility translation, jump and swap moves, and Metropolis acceptance under a common training budget.

Table S16: Improvement formulation ablation on OFFICE.

ID	configuration	J	feasible
A0	reference: difference reward, translate, $\gamma=0.995$	2\,658.5	0 %
A0'	the same with $\gamma=0$	1\,804.3	0 %
A1	+ best observed reward	2\,246.1	0 %
A2	+ best observed reward, jump and swap	2\,461.8	0 %
C1	+ Metropolis accept/reject (lattice head)	3\,067.2	48 %
C1'	the same with a continuous head	2\,737.0	0 %
A0 evaluated deterministically (argmax)		1\,445.5	0 %

S8 Completion checks on a dense instance

Figure S4 compares four completion-check settings for the same largest first constructor on one held out instance at fill ratio 0.95: no completion check, first fit, randomized packing with 32 restarts, and rollout of the base heuristic. The unchecked configuration returns an overlapping layout, while the other three complete this instance feasibly. This example illustrates geometric feasibility separately from objective value.

S9 MCNC/GSRC instance conversion

The following conversion defines the geometry and HPWL objective used in the MCNC/GSRC comparison. Block dimensions come from the GSRC `.blocks` files (`hardrectilinear` records, width and height from the corner list); netlists from the UCLA `.nets` files; terminal positions from the `.p1` files, matched to net pins by name. Pins are placed at block centers and terminals

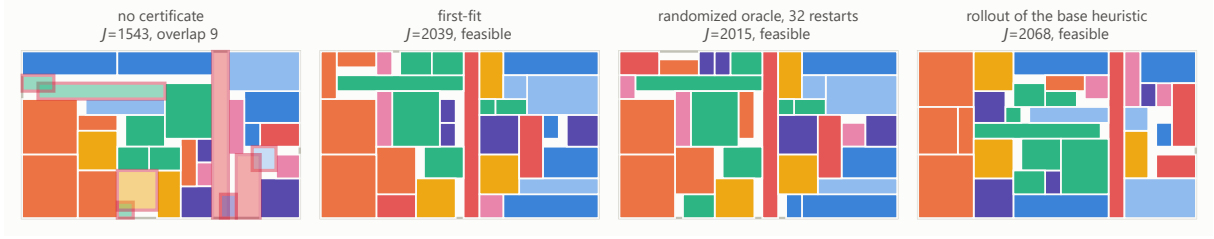


Figure S4: One dense instance under four completion-check settings of the same constructor. Overlapping facilities are outlined in red.

at their given coordinates, the standard simplification. Nets with fewer than two pins are dropped, contributing zero. Coordinates are divided by an integer for each instance and floored, which is necessary because the legality masks are dense over the plate: the factor is 6 for AMI33, 33 for AMI49 and 3 for the GSRC instances, chosen as the smallest divisor bringing the plate under 200 cells per side, and it is reported with every result. The outline is fixed by area, $\text{plate} = \text{total block area} / (1 - \text{dead space})$, at aspect ratio 1; blocks may be rotated by 90° . Under this conversion HPWL is comparable across the paper’s own methods but not against published floorplanning numbers, which use full resolution and a different pin model. Integer flooring changes block shapes and packing possibilities, so converted dead space is not treated as a published full-resolution result.

S10 Larger-instance constructor feasibility

Table S17 gives the full feasibility record of the constructors without training on 96 to 256 facilities summarized in the main text. These are constructor diagnostics; M1’s direct quality evaluation at 128 and 256 facilities is reported in S1.

Table S17: Constructors without training on larger instances, feasibility rate. Every instance has a verified feasible packing, so a failure indicates that the method did not find the existing packing. Plate geometry is drawn per cell, so methods are compared within each row. The rollout certificate is $O(n^2WH)$ per query and was not run above 128 facilities.

n	instances	no certificate	first fit	rollout
<i>fill ratio 0.85</i>				
96	15	1.00	1.00	1.00
128	15	1.00	1.00	1.00
192	15	1.00	1.00	not run
256	15	1.00	1.00	not run
<i>fill ratio 0.95</i>				
96	30	0.57	0.53	0.90
128	30	0.53	0.57	0.90
192	30	0.70	0.63	not run
256	30	0.83	0.83	not run

S11 Feasibility, initialization, and selector comparisons

These comparisons examine the search space, initialization coverage, and candidate selection separately from the primary partial-repair comparison.

Feasible search space Table S18 estimates the fraction of uniformly sampled layouts that are overlap free. This fraction decreases rapidly with size and density. Lattice sampling gives a higher feasible fraction than continuous-center sampling wherever both estimates are resolved. The comparison concerns the action representation, independently of policy training.

Table S18: Fraction of uniformly sampled layouts that are overlap free. Each entry averages four generated instances at 4×10^4 samples; $< 6 \times 10^{-6}$ means no feasible sample was drawn.

n	fill 0.35		fill 0.50		fill 0.65	
	continuous	lattice	continuous	lattice	continuous	lattice
2	2.6×10^{-1}	3.7×10^{-1}	1.7×10^{-1}	2.4×10^{-1}	7.1×10^{-2}	1.4×10^{-1}
4	3.3×10^{-2}	7.6×10^{-2}	3.0×10^{-3}	1.5×10^{-2}	5.0×10^{-4}	3.9×10^{-3}
6	4.0×10^{-3}	1.7×10^{-2}	1.6×10^{-4}	1.1×10^{-3}	$< 6 \times 10^{-6}$	2.5×10^{-5}
8	5.9×10^{-4}	4.5×10^{-3}	1.9×10^{-5}	6.9×10^{-5}	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$
10	8.8×10^{-5}	1.1×10^{-3}	$< 6 \times 10^{-6}$	1.3×10^{-5}	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$
12	6.3×10^{-6}	2.2×10^{-4}	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$
16	$< 6 \times 10^{-6}$	1.3×10^{-5}	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$	$< 6 \times 10^{-6}$

Penalty-based improvement The translation MDP uses reward $J(s_{t+1}) - J(s_t)$. With discount one, the return equals the terminal objective minus the initial objective. With discount below one, intermediate objectives also affect the return. At a common 688k-step training budget, this baseline has mean objective 2658.5 and 0% feasibility. The configuration with best-observed reward, jump and swap moves, and Metropolis acceptance reaches 3067.2 and 48% feasibility. Lattice annealing reaches 3414.8, and a masked constructor without training reaches 3380.2. Table S16 reports the individual reward and action configurations.

Initialization coverage All methods in the primary comparison receive the same verified best-contact layout, so initialization cannot explain their paired quality differences. A separate 100-instance comparison in each dense cell $(n, f) = (32, 0.90), (32, 0.95), (64, 0.95)$ gives coverage 0.78, 0.09, and 0.28 for bottom-left fill in area order, and 0.98, 0.96, and 1.00 for best-contact. These rates measure the witness generator rather than the subsequent selector.

Completion prediction The learned completion predictor uses 25,258 rollout states with an instance-grouped training and validation split. Its validation AUC is 0.902, but its best completion rate on disjoint fill 0.95 tests is 0.20, compared with 0.90 for verified first fit and 1.00 for the more expensive rollout certificate. The labels describe the outcome of one heuristic rollout, whereas a completion certificate requires an explicit feasible continuation. Predictive scores therefore do not replace feasibility verification.

Candidate selection In the full-regeneration comparison, exact marginal ranking matches or exceeds the two learned selectors in most $n = 64$ cells. On the two manually defined scenarios, masked constructive search accounts for most of the improvement over translation-based search;

learning gives a further 0.9% over the best training-free procedure in the same action set. On 60 held out moderate-density instances, beam search without training has the best paired objective and solves 57 instances, compared with 60 for the policies. These quality and coverage results support exact marginal ranking as a training-free choice for M1. Complete run-level comparisons are in the archive.

S12 Secondary methods and evaluation protocol

These experiments supplement the primary partial-repair comparison.

Design The secondary studies use six disjoint generated instance groups. The moderate-density suite crosses $n \in \{8, 16, 32, 64\}$ with $f \in \{0.40, 0.60, 0.75\}$ at 20 instances per cell (240 instances). The density comparison covers $f \in \{0.80, 0.85, 0.90, 0.95\}$ at $n = 32$ (80) and $f \in \{0.85, 0.90, 0.95\}$ at $n = 64$ (60). The large-instance constructor study uses $n \in \{96, 128, 192, 256\}$ at $f = 0.85$ and $f = 0.95$, with proportionally enlarged plates (180). A certified-selector comparison covers $n = 64, f = 0.80$ (20). The dense decoder comparison uses 100 held out instances in each $n \in \{32, 64\}$ by $f \in \{0.90, 0.95\}$ cell (400), with all decoders fixed before testing. The small-instance objective calibration uses 60 instances at $n = 8$, with plate sizes chosen for constraint-programming evaluation. These groups contain 1 040 distinct held out instances.

The two matched-time studies use separate instance groups: 200 dense and 80 boundary instances for the one-second study, and 240 instances, including 128- and 256-facility cells, for the 60-second study (Section S1). Every source packing is verified. Achieved fill is recorded for each instance; integer dimensions make the target approximate. Plate aspect ratio, types, affinities and boundary preferences vary by instance. The randomized objective constants are drawn once per suite cell.

Splits Train, validation and test draw from disjoint generator seed ranges. Instance identity is determined by the family, split, facility count, nominal fill, batch size and index, together with the stored seed and tag. The dense decoder and primary comparison families use distinct generator tags. These test families are mutually disjoint and are disjoint from training, validation and the other test cells.

S12.1 Diagnostic methods

The matched-time methods B0 to B3, B2S, M0, R0, M1, and ALNS are defined in the main-text method table. They are classical CPU implementations and share the initial layout, objective, time limit and verifier. The following methods are used in the diagnostic experiments.

- **Classical search:** shelf packing; lattice constrained simulated annealing at 8 192 and 45 000 proposals; tabu search over single facility moves; and a lattice genetic algorithm with uniform crossover and elitism. Annealing and tabu use periodic greedy polish and a common budget of 45 000 objective calls. Compared with the reference annealer, lattice annealing gives a 4.4% higher objective on OFFICE and 100% feasibility on CLINIC, against 20% for the reference. The diagnostic comparisons therefore use lattice annealing. Tabu tenure and initialization, and the GA population and memetic setting, are chosen on validation. Tabu uses a constructive start; steepest single-facility moves fail to repair random

dense starts in these tests. The memetic configuration is excluded from the matched-budget comparison because it uses ten times the evaluation budget without improving on the budget-compliant configuration.

- **Masked constructive methods without training:** greedy with largest first order; greedy with joint choice of facility and position; regret k insertion for $k \in \{2, 4, 8\}$; beam search over insertion decisions at widths 1 to 64; and beam search with the order fixed to largest first.
- **Methods without training and with certificate guidance:** largest first and joint greedy with the certificate filter of the main-text witness construction and an argmax fallback. These are empirical heuristics, not instances of the main-text feasible-continuation theorem.
- **Learned:** a constructive policy trained with PPO (Schulman et al., 2017); its heuristic variant with certificate guidance; and its full-regeneration certified variant with a carried witness. This last method uses the invariant of the main-text feasible-continuation theorem, but not the partial repair of the main-text partial-repair algorithm. Architecture and hyperparameters are in S13.

Beam width and regret k are selected on the *validation* split for the objective comparison of the main-text small-instance calibration and moderate-density diagnostic, and the test split is evaluated once with them fixed. The density cells of the density diagnostic report the whole sweep instead, because the finding there is the pattern of collapse across widths. On the two reference scenarios, single instances with no validation counterpart, the best setting is taken on the scenario itself; that favors the baselines without training, and we flag it there rather than reporting a p value.

S12.2 Policy coverage

The encoder concatenates pair features of length n , so one checkpoint serves one facility count. No single policy spans the suite, and Table S19 states exactly which instances a learned method was run on and which are comparisons of methods without training. Aggregate instance counts elsewhere in the paper refer to held out *test* instances; training and validation instances are drawn from disjoint seed ranges and are not included in the reported test aggregates.

The diagnostic experiments use the following protocol. Every method is attempted on every instance in the cells assigned to it. Objective comparisons are paired only on instances for which both methods return feasible layouts. We normalize by the best *feasible* objective achieved on that instance, because raw values differ by an order of magnitude between cells with 8 and 64 facilities. We report the mean paired gap, a 95% bootstrap confidence interval over instances, a Wilcoxon signed rank test on paired differences with Holm correction across the family, and the number of instances on which each method wins. For binary reliability we report exact binomial confidence intervals. In the prespecified dense confirmation, guided and unfiltered decoders are paired instance by instance using the exact two sided McNemar test, with Holm correction across all twelve selector and cell comparisons. Coverage and feasibility conditional on return are analyzed separately for the selective decoder.

Table S19: Datasets, and the policies evaluated on them. Training budgets and wall clock differ by policy family and are listed per checkpoint in Table S20 of S13; training cost is per policy, not amortized across rows.

dataset	n	inst.	policies	trained on
moderate density	8, 16	120	none	not used
	32	60	3 seeds, env order	train, $f \leq 0.75$
	64	60	see density rows	not used
generated, density	32	80	2 seeds, own order	train, $f=0.75$ to 0.90
	64	60	2 seeds, own order	train, $f=0.75$ to 0.90
generated, certified only	64	20	2 seeds, own order	frozen; $f=0.80$
dense decoder comparison	32, 64	400	2 seeds, own order	frozen checkpoints
one-second, dense	32, 64	200	none	not used
one-second, boundary	32, 64	80	none	not used
60-second comparison	32 to 256	240	none	not used
generated, large	96 to 256	180	none (fixed n)	not used
generated, exact	8	60	none	not used
reference scenarios	32	2	1 per scenario	that scenario
MCNC / GSRC	33 to 300	5	none (other obj.)	not used
QAPLIB	12 to 30	32	none (other obj.)	not used

S13 Policy and training

Facility tokens are embedded from features for each facility (size, placed flag, position, boundary coefficients, and the affinity and distance rows against all facilities) and mixed by self attention; a pointer head selects the facility. Position selection is a fully convolutional score over the plate from dilated residual convolutions on seven planes (occupancy, the legality mask, the affinity of the facility being placed toward whichever facility owns each cell, the boundary bonus it would earn, a mask for positions within bounds and two coordinate planes), modulated channelwise by the selected facility’s embedding. The encoder has no separate embedding for each facility index, which permits transfer across briefs at fixed n . The reward is the exact marginal of the main-text local marginal proposition, so the episode return is the objective of the completed layout up to a constant.

Unless a row of Table S20 says otherwise, every constructive policy shares one PPO configuration: $\gamma = 1$, GAE $\lambda = 0.95$, clip 0.2, Adam at 3×10^{-4} , 3 epochs per batch, minibatch 512, value coefficient 0.5, gradient norm clip 0.5, entropy coefficient 0.01 decayed linearly to zero at 80% of training with a floor of 5% of its initial value, token width 128, three attention blocks, 64 position head channels and 128 parallel environments. For every policy, the final checkpoint is used; no checkpoint selection based on validation is performed. The predictive filter is a separate supervised diagnostic: it retains the lowest validation loss state over 4 000 epochs and fixes its seven thresholds on that same grouped validation set before any test evaluation. The five improvement MDP checkpoints of the secondary formulation study (tabulated in the Online Supplement, S7) are trained separately at a matched budget of 688,128 steps (193 to 203s each). Wall clock for policy training is on one RTX 4080 SUPER; the predictive filter was trained on CPU in 17s.

The repository README maps these policy labels to the checkpoint files and training records. Seed identifiers distinguish independently trained policies; the improvement configurations retain the identifiers used in the ablation.

Table S20: Training configurations and computational cost for the learned methods. Settings that differ from the common PPO configuration are listed in the last column. Budgets end at an iteration boundary, so nominal 1.2M budgets record as 1,196,032 or 1,198,080.

policy or configuration	role	budget	min	settings
Dense-32, seeds 1/2	density, $n = 32$	1 196 032	34/50	
Dense-64, seeds 1/2	density, $n = 64$	1 196 032	34/34	
Suite, seeds 1/2/3	moderate density	1 196 032	45/44/42	random order
Transfer	transfer (S3)	1 198 080	17	64 env, 2 ep, mb 256, 48 ch
Office	OFFICE	1 998 848	34	2M steps
Clinic	CLINIC	899 072	7	0.9M steps, 64 env
A0/A0'/A1/A2/C1	improvement ablation	688 128	3.2 to 3.4	five separate runs
C1'	continuous head (S6)	688 128	2.5	continuous displacement
Completion predictor	predictive filter diagnostic	4 000 ep.	0.28	grouped validation

References

- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472. <https://doi.org/10.1287/trsc.1050.0135>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv. <https://doi.org/10.48550/arXiv.1707.06347>